

Reconsidering the `std::execution::on` algorithm

on second thought

Authors:

[Eric Niebler](#)

Date:

March 14, 2024

Source:

[GitHub](#)

Issue tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

LEWG

Synopsis

Usage experience with P2300 has revealed a gap between users' expectations and the actual behavior of the `std::execution::on` algorithm. This paper seeks to close that gap by making its behavior less surprising.

Executive Summary

Below are the specific changes this paper proposes:

1. Rename the current `std::execution::on` algorithm to `std::execution::start_on`.
2. Rename `std::execution::transfer` to `std::execution::continue_on`
3. Optional: Add a new algorithm `std::execution::on` that, like `start_on`, starts a sender on a particular context, but that remembers where execution is transitioning *from*. After the sender completes, the `on` algorithm transitions *back* to the starting execution context, giving a scoped, there-and-back-again behavior. (Alternative: don't add a new scoped `on` algorithm.)
4. Optional: Add a new uncustomizable adaptor `write_env` for writing values into the receiver's execution environment, and rename `read` to `read_env` ("read" being too vague and something of a land-grab). `write_env` is used in the implementation of the

new on algorithm and can simplify the specification of the `let_` algorithms. (Alternative: make `write_env` exposition-only.)

5. Optional: Add an uncustomizable unstopppable adaptor that is a trivial application of `write_env`: it sets the current stop token in the receiver's environment to a `never_stop_token`. `unstopppable` is used in the re-specification of the `schedule_from` algorithm. (Alternative: make `unstopppable` exposition-only.)
6. Optional: Generalize the specification for `schedule_from` to take two senders instead of a sender and a scheduler, name it `finally`, and make it uncustomizable. Specify the default implementation of `schedule_from(sch, snd)` as `finally(snd, unstopppable(schedule(sch)))`. (Alternative: keep `finally` exposition-only.)
7. Optional: Add a form of `execution::on` that lets you run part of a continuation on one scheduler, automatically transitioning back to the starting context.

Problem Description

If, knowing little about senders and sender algorithms, someone showed you code such as the following:

```
namespace ex = std::execution;

ex::sender auto work1 = ex::just()
    | ex::transfer(scheduler_A);

ex::sender auto work2 = ex::on(scheduler_B, std::move(work1))
    | ex::then([] { std::puts("hello world!"); });

ex::sender auto work3 = ex::on(scheduler_C, std::move(work2))

std::this_thread::sync_wait(std::move(work3));
```

... and asked you, which scheduler, `scheduler_A` or `scheduler_B`, is used to execute the code that prints "hello world! "? You might reasonably think the answer is `scheduler_C`. Your reasoning would go something like this:

Well clearly the first thing we execute is `on(scheduler_C, work2)`. I'm pretty sure that is going to execute `work2` on `scheduler_C`. The `printf` is a part of `work2`, so I'm going to guess that it executes on `scheduler_C`.

This paper exists because the `on` algorithm as specified in P2300R8 does *not* print "hello world!" from `scheduler_C`. It prints it from `scheduler_A`. Surprise!

But why?

`work2` executes `work1` on `scheduler_B`. `work1` then rather rudely transitions to `scheduler_A` and doesn't transition back. The `on` algorithm is cool with that. It just happily runs its continuation inline, *still on scheduler_A*, which is where "hello world!" is printed from.

If there was more work tacked onto the end of `work3`, it too would execute on `scheduler_A`.

User expectations

The authors of P2300 have witnessed this confusion in the wild. And when this author has asked his programmer friends about the code above, every single one said they expected behavior different from what is specified. This is very concerning.

However, if we change some of the algorithm names, people are less likely to make faulty assumptions about their behavior. Consider the above code with different names:

```
namespace ex = std::execution;

ex::sender auto work1 = ex::just()
    | ex::continue_on(scheduler_A);

ex::sender auto work2 = ex::start_on(scheduler_B, std::move(work1))
    | ex::then([] { std::puts("hello world!"); });

ex::sender auto work3 = ex::start_on(scheduler_C, std::move(work2))

std::this_thread::sync_wait(std::move(work3));
```

Now the behavior is a little more clear. The names `start_on` and `continue_on` both suggest a one-way execution context transition, which matches their specified behavior.

Filling the gap

on fooled people into thinking it was a there-and-back-again algorithm. We propose to fix that by renaming it to `start_on`. But what of the people who *want* a there-and-back-again algorithm?

Asynchronous work is better encapsulated when it completes on the same execution context that it started on. People are surprised, and reasonably so, if they `co_await` a task from a CPU thread pool and get resumed on, say, an OS timer thread. Yikes!

We have an opportunity to give the users of P2300 what they *thought* they were already getting, and now the right name is available: **on**.

We propose to add a new algorithm, called `on`, that remembers where execution came from and automatically transitions back there. Its operational semantics can be easily expressed in terms of the existing P2300 algorithms. It is approximately the following:

```
template <ex::scheduler Sched, ex::sender Sndr>
sender auto on(Sched sched, Sndr sndr) {
    return ex::read(ex::get_scheduler)
        | ex::let_value( [= ](auto old_sched) {
            return ex::start_on(sched, sndr)
                | ex::continue_on(old_sched);
        });
}
```

One step further?

Once we recast on as a there-and-back-again algorithm, it opens up the possibility of another there-and-back-again algorithm, one that executes a part of a *continuation* on a given scheduler. Consider the following code, where `async_read_file` and `async_write_file` are functions that return senders (description after the break):

```
ex::sender auto work = async_read_file()
  | ex::on(cpu_pool, ex::then(crunch_numbers))
  | ex::let_value([](auto numbers) {
    return async_write_file(numbers);
  });
```

Here, we read a file and then send it to an on sender. This would be a different overload of on, one that takes a sender, a scheduler, and a continuation. It saves the result of the sender, transitions to the given scheduler, and then forwards the results to the continuation, `then(crunch_numbers)`. After that, it returns to the previous execution context where it executes the `async_write_file(numbers)` sender.

The above would be roughly equivalent to:

```
ex::sender auto work = async_read_file()
  | ex::let_value([](auto numbers) {
    ex::sender auto work = ex::just(numbers)
  |
    ex::then(crunch_numbers);
    return ex::on(cpu_pool, work)
  | ex::let_value([](auto numbers) {
    return async_write_file(numbers);
  });
  });
```

This form of on would make it easy to, in the middle of a pipeline, pop over to another execution context to do a bit of work and then automatically pop back when it is done.

Implementation Experience

The perennial question: has it been implemented? It has been implemented in `stdexec` for over a year, modulo the fact that `stdexec::on` has the behavior as specified in P2300R8, and a new algorithm `exec::on` has the there-and-back-again behavior proposed in this paper.

Design Considerations

Do we really have to rename the transfer algorithm?

We don't! Within sender expressions, `work | transfer(over_there)` reads a bit nicer than `work | continue_on(over_there)`, and taken in isolation the name change is strictly for the worse.

However, the symmetry of the three operations:

- `start_on`
- `continue_on`
- `on`

... encourages developers to infer their semantics correctly. The first two are one-way transitions before and after a piece of work, respectively; the third book-ends work with transitions. In the author's opinion, this consideration outweighs the other.

Do we need the additional form of `on`?

We don't! Users can build it themselves from the other pieces of P2300 that will ship in C++26. But the extra overload makes it much simpler for developers to write well-behaved asynchronous operations that complete on the same execution contexts they started on, which is why it is included here.

What happens if there's no scheduler for `on` to go back to?

If we recast `on` as a there-and-back-again algorithm, the implication is that the receiver that gets connect-ed to the `on` sender must know the current scheduler. If it doesn't, the code will not compile because there is no scheduler to go back to.

Passing an `on` sender to `sync_wait` will work because `sync_wait` provides a `run_loop` scheduler as the current scheduler. But what about algorithms like `start_detached` and `spawn` from [P3149](#)? Those algorithms connect the input sender with a receiver whose environment lacks a value for the `get_scheduler` query. As specified in this paper, those algorithms will reject `on` senders, which is bad from a usability point of view.

There are a number of possible solutions to this problem:

1. Any algorithm that eagerly connects a sender should take an environment as an optional extra argument. That way, users have a way to tell the algorithm what the current scheduler is. They can also pass additional information like allocators and stop tokens.
2. Those algorithms can specify a so-called "inline" scheduler as the current scheduler, essentially causing the `on` sender to perform a no-op transition when it completes.
3. Those algorithms can treat top-level `on` senders specially by converting them to `start_on` senders.
4. Those algorithms can set a hidden, non-forwarding "root" query in the environment. The `on` algorithm can test for this query and, if found, perform a no-op transition when it completes. This has the advantage of not setting a "current" scheduler, which could interfere with the behavior of nested senders.

The author of this paper likes options (1) and (4), and will be writing a paper proposing both of these changes.

Questions for LEWG's consideration

The author would like LEWG's feedback on the following two questions:

1. If `on` is renamed `start_on`, do we also want to rename `transfer` to `continue_on`?
2. If `on` is renamed `start_on`, do we want to add a new algorithm named `on` that book-ends a piece of work with transitions to and from a scheduler?
3. If we want the new scoped form of `on`, do we want to add the `on(sndr, sched, continuation)` algorithm overload to permit scoped execution of continuations?
4. Do we want to make the `write_env` adaptor exposition-only, or make it public?
5. Do we want to make the `unstoppable` adaptor exposition-only, or make it public?
6. Do we want to make the `finally` algorithm an exposition-only detail of the `schedule_from` algorithm, or make it public?

Proposed Wording

[Editorial note: The wording in this section is based on [P2300R8](#) with the addition of [P8255R1](#). -- end note]

Change `[exec.syn]` as follows:

```
    inline constexpr unspecified read_env{};
    ...

    struct start_on_t;
    struct transfer_tcontinue_on_t;
    struct on_t;
    struct schedule_from_t;
    ...

    inline constexpr unspecified write_env{};
    inline constexpr unspecified unstoppable{};
    inline constexpr start_on_t start_on{};
    inline constexpr transfer_ttransfercontinue_on_t
continue_on{};
    inline constexpr on_t on{};
    inline constexpr unspecified finally{};
    inline constexpr schedule_from_t schedule_from{};
```

Change subsection “`execution::read` [`exec.read`]” to “`execution::read_env` [`exec.read.env`]”, and within that subsection, replace every instance of “`read`” with “`read_env`”.

After `[exec.adapt.objects]`, add a new subsection “`execution::write_env` [`exec.write.env`]” as follows:

execution::write_env [exec.write.env]

1. `write_env` is a sender adaptor that connects its inner sender with a receiver that has the execution environment of the outer receiver joined with a specified execution environment.
2. `write_env` is a customization point object. For some subexpressions `sndr` and `env`, if `decltype((sndr))` does not satisfy sender or if `decltype((env))` does not satisfy queryable, the expression `write_env(sndr, env)` is ill-formed. Otherwise, it is expression-equivalent to `make_sender(write_env, env, sndr)`.
3. The exposition-only class template `impls-for` ([exec.snd.general]) is specialized for `write_env` as follows:

```
template<>
struct impls-for<tag_t<write_env>> : default-impls {
    static constexpr auto get-env =
        [](auto, const auto& state, const auto& rcvr) noexcept {
            return JOIN-ENV(state, get_env(rcvr));
        };
};
```

After [exec.write.env], add a new subsection “`execution::unstoppable` [exec.unstoppable]” as follows:

execution::unstoppable [exec.unstoppable]

1. `unstoppable` is a sender adaptor that connects its inner sender with a receiver that has the execution environment of the outer receiver but with a `never_stop_token` as the value of the `get_stop_token` query.
2. For a subexpression `sndr`, `unstoppable(sndr)` is expression equivalent to `write_env(sndr, MAKE-ENV(get_stop_token, never_stop_token{}))`.

Change subsection “`execution::on` [exec.on]” to “`execution::start_on` [exec.start.on]”, and within that subsection, replace every instance of “`on`” with “`start_on`” and every instance of “`on_t`” with “`start_on_t`”.

Change subsection “`execution::transfer` [exec.transfer]” to “`execution::continue_on` [exec.complete.on]”, and within that subsection, replace every instance of “`transfer`” with “`continue_on`” and every instance of “`transfer_t`” with “`continue_on_t`”.

Change subsection “`execution::schedule_from` [exec.schedule.from]” to “`execution::finally` [exec.finally]”, change every instance of “`schedule_from`” to “`finally`” and “`schedule_from_t`” to “`tag_t<finally>`”, and change the subsection as follows:

execution::finally [exec.finally]

[Editorial note: Replace paragraphs 1-3 with the following: -- end note]

1. `finally` is a sender adaptor that starts one sender unconditionally after another sender completes. If the second sender completes successfully, the `finally` sender completes with the async results of the first sender. If the second sender completes with error or stopped, the async results of the first sender are discarded, and the `finally` sender completes with the async results of the second sender. *[Note: It is similar in spirit to the try/finally control structure of some languages. -- end note]*
2. The name `finally` denotes a customization point object. For some subexpressions `try_sndr` and `finally_sndr`, if `try_sndr` or `finally_sndr` do not satisfy sender, the expression `finally(try_sndr, finally_sndr)` is ill-formed; otherwise, it is expression-equivalent to `make_sender(finally, {}, try_sndr, finally_sndr)`.
3. Let `CS` be a specialization of `completion_signatures` whose template parameters are the pack `Sigs`. Let `VALID-FINALLY(CS)` be true if and only if there is no type in `Sigs` of the form `set_value_t(Ts...)` for which `sizeof...(Ts)` is greater than 0. Let `F` be `decltype((finally_sndr))`. If `sender_in<F>` is true and `VALID-FINALLY(completion_signatures_of_t<F>)` is false, the program is ill-formed.
4. The exposition-only class template `impls-for` (`[exec.snd.general]`) is specialized for `finally` as follows:

```
template<>
struct impls-for<tag_t<finally>> : default-impls {
    static constexpr auto get-attrs = see below;
    static constexpr auto get-state = see below;
    static constexpr auto complete = see below;
};
```

1. The member `impls-for<tag_t<finally>>::get-attrs` is initialized with a callable object equivalent to the following lambda:

```
[(const auto& data, const auto& child) noexcept ->
decltype(auto) {
    return JOIN-ENV(SCHED-ATTRS(data), FWD-
ENV(get_env(child)));
}]

[(auto, const auto& tsndr, const auto& fsndr) noexcept
-> decltype(auto) {
    return JOIN-ENV(FWD-ENV(get_env(fsndr)), FWD-
ENV(get_env(tsndr)));
}]
```

2. The member `impls-for<tag_t<finally>>::get-state` is initialized with a callable object equivalent to the following lambda:

```
[<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr)
requires sender_in<child-type<Sndr, 0>,
env_of_t<Rcvr>> &&
sender_in<child-type<Sndr, 1>, env_of_t<Rcvr>> &&
VALID-FINALLY(completion_signatures_of_t<child-
type<Sndr, 1>, env_of_t<Rcvr>>) {
```

```

return apply(
{&}<class Sch, class Child>(auto, Sch sch, Child&&
child)
[&]<class TSndr, class FSndr>(auto, auto, TSndr&&
tsndr, FSndr&& fsndr) {
    using variant-type = see below;
    using receiver-type = see below;
    using operation-type =
connect_result_t<schedule_result_t<Sch>FSndr, receiver-
type>;

    struct state-type {
        Rcvr& rcvr;
        variant-type async-result;
        operation-type op-state;

        explicit state-type(Sch schFSndr&& fsndr, Rcvr&
rcvr)
            : rcvr(rcvr)
            , op-
state(connect(schedule(sch)std::forward<FSndr>(fsndr),
receiver-type{{}, this}))
    {}

        };

    return state-type{schstd::forward<FSndr>(fsndr),
rcvr};
    },
    std::forward<Sndr>(sndr));
}

```

1. The local class *state-type* is a structural type.
2. Let *Sigs* be a pack of the arguments to the completion_signatures specialization *named* by completion_signatures_of_t<~~Child~~TSndr, env_of_t<Rcvr>>. Let *as-tuple* be an alias template that transforms a completion signature Tag(Args...) into the tuple specialization *decayed-tuple*<Tag, Args...>. Then *variant-type* denotes the type variant<monostate, *as-tuple*<*Sigs*>...>, except with duplicate types removed.
3. Let *receiver-type* denote the following class:

```

struct receiver-type : receiver_adaptor<receiver-
type> {
    state-type* state; // exposition only

    Rcvr&& base() && noexcept { return std::move(state-
>rcvr); }
    const Rcvr& base() const & noexcept { return state-
>rcvr; }

    void set_value() && noexcept {
        visit(
            [this]<class Tuple>(Tuple& result) noexcept ->
void {
                if constexpr (!same_as<monostate, Tuple>) {
                    auto& [tag, ...args] = result;
                    tag(std::move(state->rcvr),

```

```

        std::move(args)...);
    }
    },
    state->async-result);
}
};

```

3. The member *impls-for*<tag_t<finally>>::*complete* is initialized with a callable object equivalent to the following lambda:

```

[]<class Tag, class... Args>(auto, auto& state, auto&
rcvr, Tag, Args&&... args) noexcept -> void {
    using result_t = decayed-tuple<Tag, Args...>;
    constexpr bool nothrow =
is_nothrow_constructible_v<result_t, Tag, Args...>;

    TRY-EVAL(std::move(rcvr), [&]() noexcept(nothrow) {
        state.async-result.template emplace<result_t>(Tag(),
std::forward<Args>(args)...);
    }());

    if (state.async-result.valueless_by_exception())
        return;
    if (state.async-result.index() == 0)
        return;

    start(state.op-state);
};

```

[Editorial note: Remove paragraph 5, which is about the requirements on customizations of the algorithm; finally cannot be customized. -- end note]

Insert a new subsection “*execution::schedule_from* [exec.schedule.from]” as follows:

execution::schedule_from [exec.schedule.from]

[Editorial note: These three paragraphs are taken unchanged from P2300R8. -- end note]

1. *schedule_from* schedules work dependent on the completion of a sender onto a scheduler’s associated execution resource. *schedule_from* is not meant to be used in user code; it is used in the implementation of transfer.
2. The name *schedule_from* denotes a customization point object. For some subexpressions *sch* and *snr*, let *Sch* be *decltype*((*sch*)) and *Snr* be *decltype*((*snr*)). If *Sch* does not satisfy scheduler, or *Snr* does not satisfy sender, *schedule_from* is ill-formed.
3. Otherwise, the expression *schedule_from*(*sch*, *snr*) is expression-equivalent to:

```

transform_sender(
    query-or-default(get_domain, sch, default_domain()),
    make_sender(schedule_from, sch, snr));

```

4. The exposition-only class template *impls-for* is specialized for *schedule_from_t* as follows:

```
template<>
struct impls-for<schedule_from_t> : default-impls {
    static constexpr auto get_attrs =
        [](const auto& data, const auto& child) noexcept ->
decltype(auto) {
    return JOIN-ENV(SCHED-ATTRS(data), FWD-
ENV(get_env(child)));
};
};
```

5. Let *sndr* and *env* be subexpressions such that *Sndr* is *decltype((sndr))*. If *sender-for<Sndr, schedule_from_t>* is false, then the expression *schedule_from.transform_sender(sndr, env)* is ill-formed; otherwise, it is equal to:

```
auto&& [tag, sch, child] = sndr;
return finally(std::forward_like<Sndr>(child),
               unstopable(schedule(std::forward_like<Sndr>
(sch))));
```

This causes the *schedule_from(sch, sndr)* sender to become *finally(sndr, unstopable(schedule(sch)))* when it is connected with a receiver with an execution domain that does not customize *schedule_from*.

[Editorial note: The following paragraph is taken unchanged from P2300R8. -- end note]

6. Let the subexpression *out_sndr* denote the result of the invocation *schedule_from(sch, sndr)* or an object copied or moved from such, and let the subexpression *rcvr* denote a receiver such that the expression *connect(out_sndr, rcvr)* is well-formed. The expression *connect(out_sndr, rcvr)* has undefined behavior unless it creates an asynchronous operation ([*async.ops*]) that, when started:

- eventually completes on an execution agent belonging to the associated execution resource of *sch*, and
- completes with the same *async* result as *sndr*.

Insert a new subsection “*execution::on [exec.on]*” as follows:

execution::on [exec.on]

1. The *on* sender adaptor has two forms:

- one that starts a sender *sndr* on an execution agent belonging to a particular scheduler’s associated execution resource and that restores execution to the starting execution resource when the sender completes, and

- one that, upon completion of a sender `sndr`, transfers execution to an execution agent belonging to a particular scheduler's associated execution resource, then executes a sender adaptor closure with the async results of the sender, and that then transfers execution back to the execution resource `sndr` completed on.
2. The name `on` denotes a customization point object. For some subexpressions `sch` and `sndr`, if `decltype((sch))` does not satisfy `scheduler`, or `decltype((sndr))` does not satisfy `sender`, `on(sch, sndr)` is ill-formed.
 3. Otherwise, the expression `on(sch, sndr)` is expression-equivalent to:

```
transform_sender(
    query-or-default(get_domain, sch, default_domain()),
    make_sender(on, sch, sndr));
```

4. For a subexpression `closure`, if `decltype((closure))` is not a sender adaptor closure object ([`exec.adapt.objects`]), the expression `on(sndr, sch, closure)` is ill-formed; otherwise, it is equivalent to:

```
transform_sender(
    get_domain_early(sndr),
    make_sender(on, pair{sch, closure}, sndr));
```

5. Let `out_sndr` and `env` be subexpressions such that `OutSndr` is `decltype((out_sndr))`. If `sender-for<OutSndr, on_t>` is false, then the expressions `on.transform_env(out_sndr, env)` and `on.transform_sender(out_sndr, env)` are ill-formed; otherwise:

1. Let `none-such` be an unspecified empty class type and let `not-a-sender` be the exposition-only type:

```
struct not-a-sender {
    using sender_concept = sender_t;

    auto get_completion_signatures(auto&&) const {
        return see below;
    }
};
```

... where the member function `get_completion_signatures` returns an object of a type that is not a specialization of the `completion_signatures` class template.

2. `on.transform_env(out_sndr, env)` is equivalent to:

```
auto&& [ign1, data, ign2] = out_sndr;
if constexpr (scheduler<decltype(data)>) {
    return JOIN-ENV(SCHED-ENV(data), FWD-ENV(env));
} else {
    using Env = decltype((env));
    return static_cast<remove_rvalue_reference_t<Env>>
(std::forward<Env>(env));
}
```

3. `on.transform_sender(out_sndr, env)` is equivalent to:

```
auto&& [ign, data, sndr] = out_sndr;
if constexpr (scheduler<decltype(data)>) {
    auto old_sch =
        query-with-default(get_scheduler, env, none-such{});

    if constexpr (same_as<decltype(old_sch), none-such>) {
        return not-a-sender{};
    } else {
        return start_on(std::forward_like<OutSndr>(data),
            std::forward_like<OutSndr>(sndr))
            | continue_on(std::move(old_sch));
    }
} else {
    auto&& [sch, closure] = std::forward_like<OutSndr>
(data);
    auto old_sch = query-with-default(
        get_completion_scheduler<set_value_t>,
        get_env(sndr),
        query-with-default(get_scheduler, env, none-such{}));

    if constexpr (same_as<decltype(old_sch), none-such>) {
        return not-a-sender{};
    } else {
        return std::forward_like<OutSndr>(sndr)
            | write-env(SCHED-ENV(old_sch));
            | continue_on(sch)
            | std::forward_like<OutSndr>(closure)
            | continue_on(old_sch)
            | write-env(SCHED-ENV(sch));
    }
}
```

4. *Recommended practice:* Implementations should use the return type of `not-a-sender::get_completion_signatures` to inform users that their usage of `on` is incorrect because there is no available scheduler onto which to restore execution.

[Editorial note: The following changes to the `let_*` algorithms are not strictly necessary; they are simplifications made possible by the addition of the `write_env` adaptor above. -- end note]

Remove [exec.let]p5.1, which defines an exposition-only class *receiver2*.

Change [exec.let]p5.2.2 as follows:

2. Let *as-sndr2* be an alias template such that *as-sndr2*<Tag(Args...)> denotes the type `call-result-t<tag t<write env>, call-result-t<Fn, decay_t<Args>&...>, Env>`. Then *ops2-variant-type* denotes the type `variant<monostate, connect_result_t<as-sndr2<LetSigs>, receiver2<Rcvr, Env>>...>`.

Change [exec.let]p5.3 as follows:

3. The exposition-only function template *let-bind* is ~~equal to~~ as follows:

```

template<class State, class Rcvr, class... Args>
void let_bind(State& state, Rcvr& rcvr, Args&&... args) {
    auto& args = state.args.emplace<decayed_tuple<Args...>>
(std::forward<Args>(args)...);
    auto sndr2 = write_env(apply(std::move(state.fn), args), __
std::move(state.env)); // see [exec.adapt.general]
auto rcvr2 = receiver2{std::move(rcvr),
std::move(state.env)};
    auto mkop2 = [&] { return connect(std::move(sndr2),
std::move(rcvr2)); };
    auto& op2 = state.ops2.emplace<decltype(mkop2())>(emplace-
from{mkop2});
    start(op2);
}

```

Acknowledgments

I'd like to thank my dog, Luna.